Sistemas de bancos de dados

Disciplina:

Sistemas de bancos de dados

Conteúdo:

- Visões, índices, constraints e noções de transações.
- Álgebra relacional e introdução ao cálculo relacional.

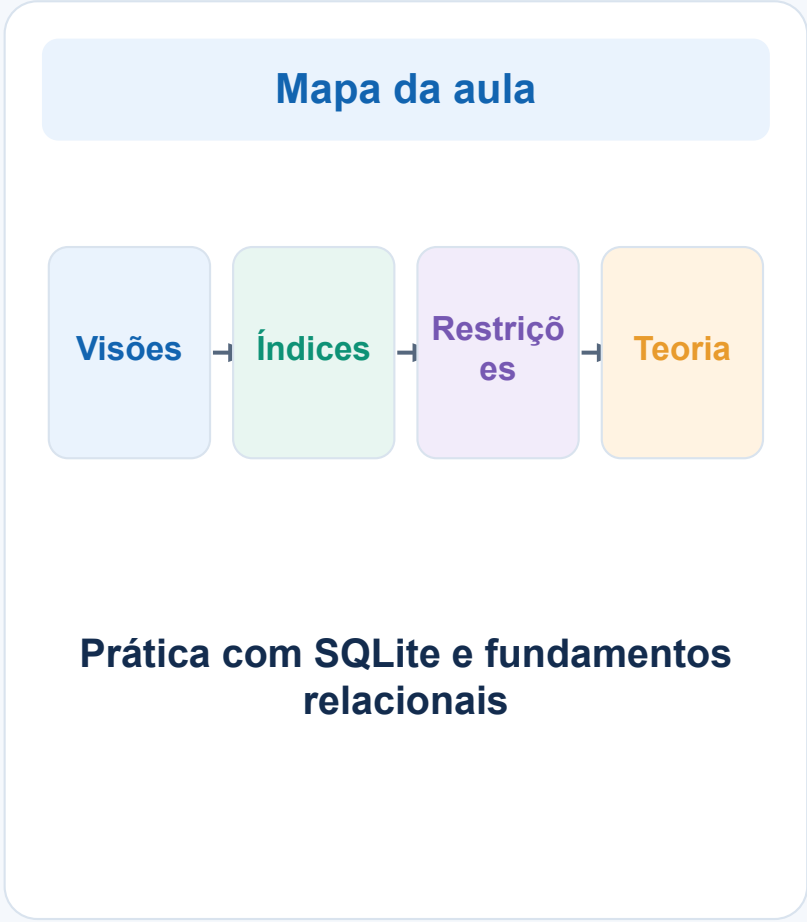
Ferramentas recomendadas:

DBeaver e SQlite.

Visões, índices e transações

Tópico: Da organização dos dados à lógica das consultas

Nesta aula, ampliaremos o uso do SQL para organizar consultas, melhorar buscas e proteger a consistência dos registros em bancos relacionais. Também conheceremos a álgebra e o cálculo relacional, duas bases teóricas que ajudam a compreender como as consultas expressam perguntas sobre os dados.



Objetivos de aprendizagem

Tópico: O que o estudante deverá conseguir fazer

Ao final, o estudante deverá criar uma visão simples, definir um índice e reconhecer situações em que ele pode auxiliar uma consulta. Deverá também aplicar restrições de integridade, demonstrar o efeito de confirmar ou desfazer uma transação e interpretar comandos básicos.

Resultados esperados

1. Criar e consultar uma visão
2. Examinar um plano de busca
3. Testar regras de integridade
4. Explicar álgebra e cálculo

Estudo de caso: pequena loja

Tópico: Dados fictícios para os exemplos práticos

O laboratório utilizará duas tabelas: clientes, com identificação e localização, e pedidos, com valor e situação de cada compra. Os dados foram criados exclusivamente para ensino; seus valores simples facilitam conferir manualmente os resultados das consultas e das restrições durante a aula.

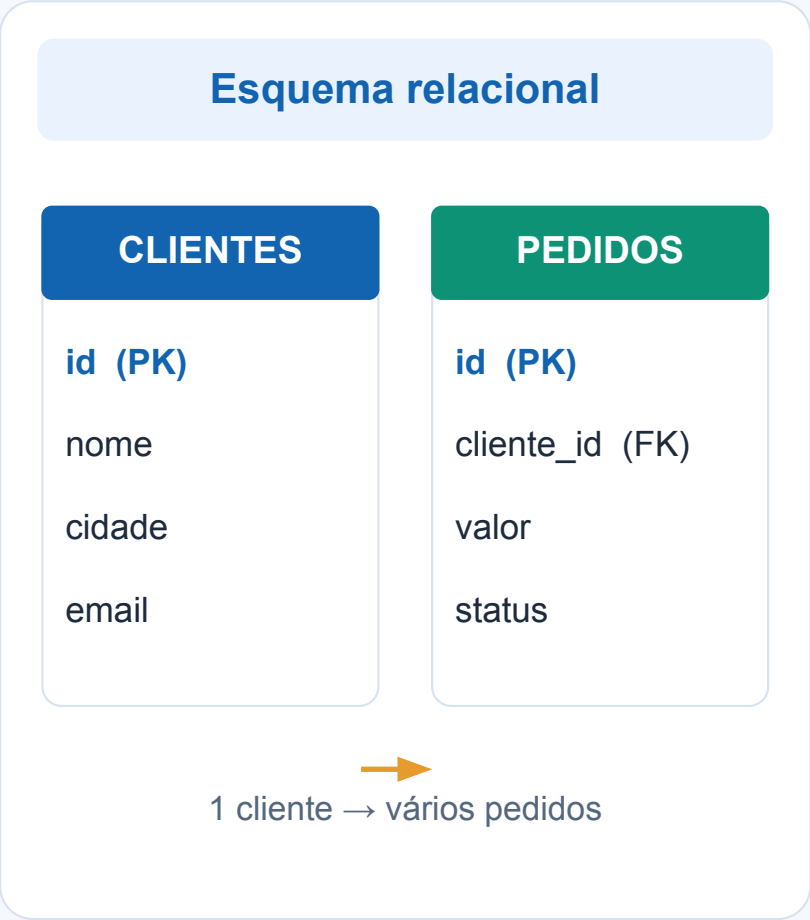
Amostra dos pedidos

id	cliente	valor	status
101	Ana	120	pago
102	Ana	80	aberto
103	Bruno	200	pago
105	Carla	150	pago

Tabela ilustrativa: pedidos ligados a clientes

Relacionamento entre tabelas

Tópico: Chaves primárias e estrangeiras no modelo



A chave primária identifica cada linha sem ambiguidade: clientes.id e pedidos.id são exemplos de identificadores únicos dentro de suas tabelas. Essa ligação permite reunir informações em consultas e também impedir, quando a restrição está ativa, pedidos associados a clientes inexistentes.

Preparando DBeaver e SQLite

Tópico: Abrir uma conexão e executar um roteiro

No DBeaver, crie uma conexão do tipo SQLite e indique o caminho de um arquivo local destinado exclusivamente às atividades da disciplina. Abra um novo editor SQL associado à conexão e execute primeiro a configuração de chaves estrangeiras, antes de iniciar transações.

Passos no DBeaver

1. Nova conexão → SQLite
2. Selecionar arquivo .db de testes
3. Abrir editor SQL
4. Executar PRAGMA foreign_keys = ON

Criando o esquema no SQLite

Tópico: Estrutura com tipos, chaves e regras

O comando CREATE TABLE define o nome de uma tabela, suas colunas e as regras que deverão ser respeitadas durante alterações. Neste exemplo, clientes.id e pedidos.id são identificadores inteiros; pedidos.valor precisa ser positivo, e cliente_id deve apontar para um cliente válido.

Trecho do esquema SQL

```
CREATE TABLE clientes (
 id INTEGER PRIMARY KEY,
 nome TEXT NOT NULL,
 cidade TEXT NOT NULL,
 email TEXT UNIQUE
);

CREATE TABLE pedidos (
 id INTEGER PRIMARY KEY,
 cliente_id INTEGER NOT NULL,
 valor REAL CHECK(valor > 0),
 FOREIGN KEY(cliente_id)
 REFERENCES clientes(id)
);
```

Inserindo dados de exemplo

Tópico: Registros que serão usados na aula

Após criar as tabelas, o comando INSERT INTO adiciona linhas com os valores necessários para testar consultas e restrições de integridade. Inseriremos quatro clientes e seis pedidos, com situações como pago, aberto e cancelado; um cliente ficará propositalmente sem compras.

Amostra de inserção

```
INSERT INTO clientes
(id,nome,cidade,email) VALUES
(1,'Ana','Vitória',
 'ana@exemplo.com');

INSERT INTO pedidos
(id,cliente_id,valor,status)
VALUES
(101,1,120,'pago');
```

Conferindo o banco de testes

Tópico: Primeira verificação antes de avançar

Conferência inicial

```
SELECT *
FROM clientes
ORDER BY id;

SELECT *
FROM pedidos
ORDER BY id;

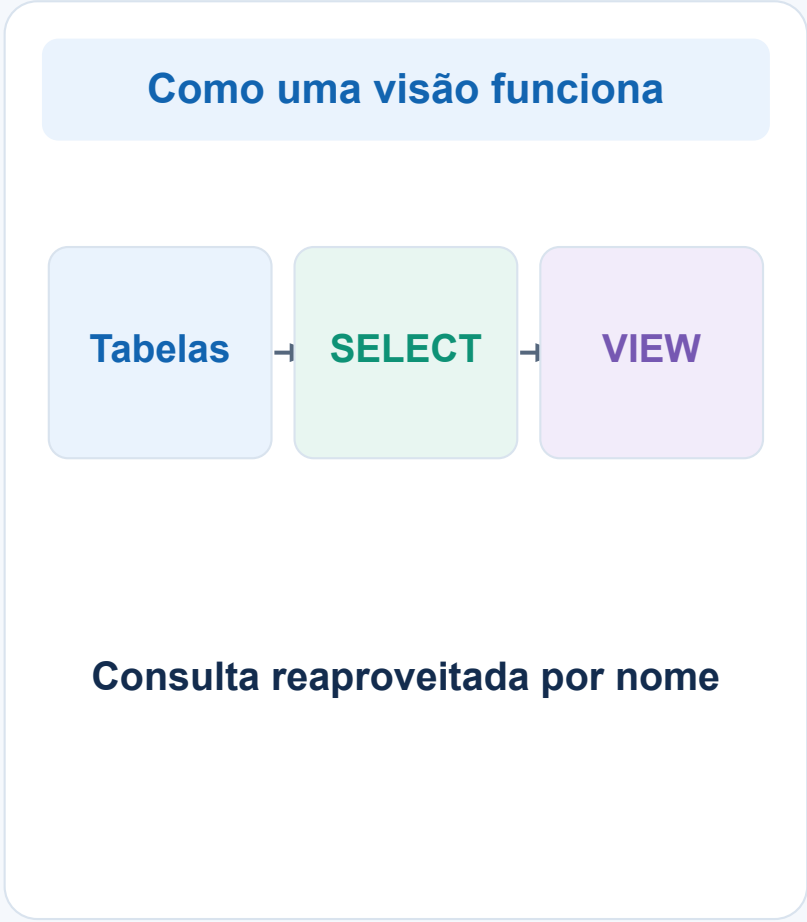
SELECT COUNT(*)
FROM pedidos;
```

Uma boa prática de laboratório é verificar a carga inicial antes de criar novos objetos, evitando interpretar erros de preparação como erros conceituais. Execute SELECT nas duas tabelas, confira a quantidade de registros e observe se os identificadores e os estados dos pedidos estão corretos.

O que é uma visão?

Tópico: Consulta nomeada para reutilização

Uma visão, chamada *view* em inglês, é uma consulta SELECT armazenada sob um nome, permitindo acessá-la como se fosse uma tabela. Em vez de repetir uma junção e um filtro em várias consultas, criamos uma definição única que pode ser reutilizada.



Criando uma visão com *JOIN*

Tópico: Pedidos pagos com dados dos clientes

A instrução CREATE *VIEW* associa um nome à consulta que reúne os pedidos com seus respectivos clientes e filtra somente as compras pagas. O operador *JOIN* liga as tabelas pelo identificador do cliente, enquanto WHERE define quais pedidos devem aparecer na visão.

CREATE VIEW na prática

```
CREATE VIEW vw_pedidos_pagos AS
SELECT p.id AS pedido_id,
       c.nome AS cliente,
       c.cidade, p.valor
FROM pedidos AS p
JOIN clientes AS c
 ON c.id = p.cliente_id
WHERE p.status = 'pago';
```

Consultando a visão criada

Tópico: SELECT sobre uma consulta reutilizável

Depois de criada, a visão pode aparecer na cláusula FROM de outra consulta, exatamente como acontece com uma tabela convencional. No banco de exemplo, a visão retorna três pedidos pagos: Ana com 120, Bruno com 200 e Carla com 150 unidades monetárias.

Consultas na visão

```
SELECT *
FROM vw_pedidos_pagos
ORDER BY valor DESC;

SELECT cliente, SUM(valor)
FROM vw_pedidos_pagos
GROUP BY cliente;
```

Limites e cuidados com visões

Tópico: Simplificação não significa cópia dos dados

No SQLite, INSERT, UPDATE e DELETE não alteram diretamente uma visão comum; há exceções avançadas usando gatilhos do tipo INSTEAD OF. Caso a definição precise mudar, avalie recriar a visão e verificar se outras consultas dependem de seus nomes e colunas.

Tabela × visão

Tabela

Armazena registros

Visão

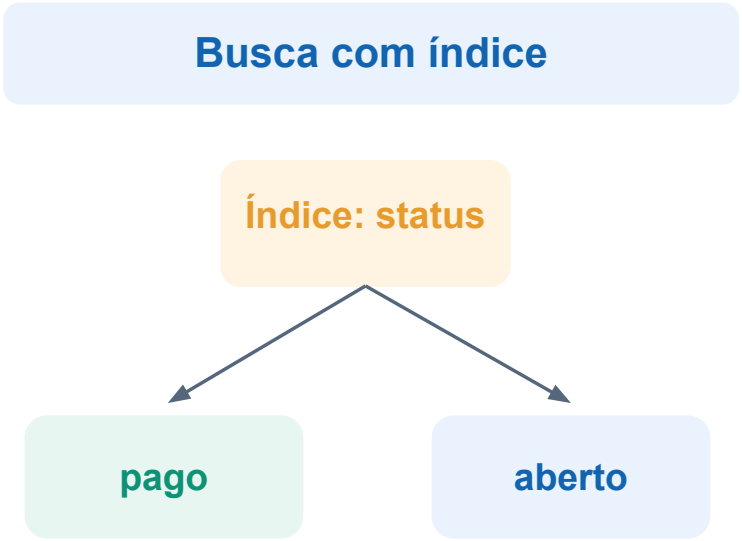
Armazena definição SELECT

Índice

Estrutura de acesso auxiliar

O que é um índice?

Tópico: Estrutura auxiliar para localizar registros



Caminho de busca ilustrativo

Um índice é uma estrutura criada sobre uma ou mais colunas para ajudar o mecanismo do banco a localizar registros com menos trabalho. A ideia lembra o índice de um livro: em vez de ler todas as páginas, usamos uma organização auxiliar para encontrar um assunto.

Criando um índice no SQLite

Tópico: CREATE *INDEX* sobre a coluna status

O comando CREATE *INDEX* define uma estrutura de acesso auxiliar e informa qual tabela e coluna serão consideradas durante a busca. Após criar o índice, a tabela e os resultados das consultas continuam os mesmos; o que pode mudar é a estratégia de execução.

Criar e remover índice

```
CREATE INDEX idx_pedidos_status
ON pedidos(status);

SELECT id, status
FROM pedidos
WHERE status = 'pago';

-- Para remover:
DROP INDEX idx_pedidos_status;
```

Inspecionando o plano de busca

Tópico: EXPLAIN *QUERY PLAN* no laboratório

A indicação *SCAN* normalmente representa uma varredura, enquanto *SEARCH* mostra uma busca por um subconjunto de linhas, frequentemente com índice. Em tabelas muito pequenas, o otimizador pode preferir *SCAN* mesmo que exista índice; compare estratégias, sem prometer ganho de tempo.

Interpretando o plano

SCAN pedidos
Percorre registros da tabela

SEARCH pedidos USING INDEX
Pode localizar subconjunto pelo índice

Atenção ao otimizador
A escolha varia com dados e estatísticas

Quando índices ajudam ou atrapalham

Tópico: Custo de leitura, escrita e armazenamento

Por outro lado, cada INSERT, UPDATE ou DELETE pode exigir manutenção da estrutura indexada, além do espaço ocupado em disco. Portanto, criar índices indiscriminadamente não é uma boa prática: observe consultas reais, volume de dados e planos antes de decidir.

Trade-offs dos índices

Benefício

Pode reduzir leituras desnecessárias

Custo

Ocupa espaço e exige atualização

Decisão

Depende da consulta e dos dados

O que são *constraints*?

Tópico: Regras para preservar integridade dos dados

Regras mais usadas

PRIMARY KEY
Identifica cada linha

FOREIGN KEY
Relaciona registros de tabelas

CHECK / NOT NULL
Controla valores e preenchimento

UNIQUE / DEFAULT
Evita duplicidade / define padrão

As restrições, chamadas *constraints*, definem condições que devem ser respeitadas quando registros são inseridos ou modificados em uma tabela. Essas verificações complementam a validação feita pelas aplicações, pois protegem os dados mesmo quando diferentes programas acessam o mesmo arquivo SQLite.

PRIMARY KEY, UNIQUE e *NOT NULL*

Tópico: Identificação, unicidade e preenchimento

UNIQUE evita repetição de valores não nulos na coluna email, enquanto *NOT NULL* exige que um campo receba um valor diferente de NULL. Essas regras têm finalidades distintas: uma pessoa pode ter nome obrigatório, email único e identificador principal, sem que os conceitos se confundam.

Regras na tabela clientes

```
CREATE TABLE clientes (
 id INTEGER PRIMARY KEY,
 nome TEXT NOT NULL,
 cidade TEXT NOT NULL,
 email TEXT UNIQUE
);

-- id identifica a linha
-- nome é obrigatório
-- email não se repete
```

FOREIGN KEY no SQLite

Tópico: Integridade referencial entre clientes e pedidos

FOREIGN KEY expressa que o valor de pedidos.cliente_id deve corresponder ao identificador de um cliente existente, salvo regras específicas de nulidade. Execute PRAGMA foreign_keys = ON antes de iniciar a transação e consulte seu valor; o retorno 1 indica fiscalização ativada.

Habilitar e conferir

```
PRAGMA foreign_keys = ON;
PRAGMA foreign_keys;

CREATE TABLE pedidos (
 id INTEGER PRIMARY KEY,
 cliente_id INTEGER NOT NULL,
 FOREIGN KEY (cliente_id)
  REFERENCES clientes(id)
);
```

CHECK e *DEFAULT*

Tópico: Validar valores e definir preenchimento padrão

CHECK impede valores que violem uma condição, como pedido com valor menor ou igual a zero ou situação fora das opções permitidas. *DEFAULT* fornece um valor inicial quando a coluna é omitida no INSERT; em nosso exemplo, o status padrão será aberto.

Regras em pedidos

```
valor REAL NOT NULL
  CHECK (valor > 0),

status TEXT NOT NULL
  DEFAULT 'aberto'
  CHECK (status IN (
    'aberto',
    'pago',
    'cancelado'
  ))
```

Testando violações de regras

Tópico: Aprender observando mensagens de erro

Para comprovar o funcionamento das restrições, tente inserir um pedido associado ao cliente 999, que não está cadastrado no banco de testes. As mensagens de erro demonstram que o banco rejeita dados inválidos; não inclua esses comandos no roteiro de preparação executado integralmente.

Comandos para testar isolados

```
-- Chave estrangeira inválida
INSERT INTO pedidos
(id,cliente_id,valor)
VALUES (107,999,40);

-- CHECK inválido
INSERT INTO pedidos
(id,cliente_id,valor)
VALUES (108,1,-10);
```

O que é uma transação?

Tópico: Agrupar operações em uma unidade lógica



Uma transação reúne uma ou mais operações que devem ser tratadas como uma unidade, evitando alterações parciais em situações de falha. Por exemplo, registrar uma compra pode exigir inserir o pedido e atualizar informações relacionadas; se uma etapa falhar, talvez seja necessário desfazer tudo.

BEGIN, *COMMIT* e *ROLLBACK*

Tópico: Iniciar, confirmar ou desfazer alterações

COMMIT confirma as alterações realizadas com sucesso, enquanto *ROLLBACK* desfaz as modificações ainda não confirmadas naquela transação. No DBeaver, observe o modo de confirmação automática da conexão antes do exercício, pois ele pode influenciar a experiência ao executar comandos separados.

Exemplo com desfazer

```
BEGIN TRANSACTION;

INSERT INTO pedidos
(id,cliente_id,valor,status)
VALUES (107,4,75,'aberto');

ROLLBACK;

SELECT * FROM pedidos
WHERE id = 107;
```

Propriedades ACID

Tópico: Quatro ideias fundamentais de transações

Atomicidade significa tudo ou nada; consistência envolve respeitar regras; isolamento trata da interação entre transações concorrentes; durabilidade preserva mudanças confirmadas. Essas propriedades ajudam a entender por que operações de banco não devem ser tratadas como simples gravações de texto sem controle.

O significado de ACID

A — **Atomicidade**
Tudo ou nada

C — **Consistência**
Regras preservadas

I — **Isolamento**
Transações concorrentes

D — **Durabilidade**
Confirmado persiste

Prática: confirmar ou desfazer

Tópico: Comparar o estado do banco antes e depois

Primeiro, execute uma transação que insere um pedido de Diego e termina com *ROLLBACK*; consulte a tabela para comprovar que ele desapareceu. Compare os resultados e observe que a visão de pedidos pagos passa a incluir a compra confirmada, porque consulta as tabelas atualizadas.

Exemplo de confirmação

```
BEGIN TRANSACTION;
UPDATE pedidos
SET status = 'pago'
WHERE id = 102;
COMMIT;

SELECT *
FROM vw_pedidos_pagos;
```

Por que estudar álgebra relacional?

Tópico: Operações matemáticas sobre relações

A álgebra relacional é um conjunto de operações formais usadas para descrever transformações e combinações de relações, que podemos associar a tabelas. As operações de seleção, projeção e junção têm correspondências intuitivas com WHERE, SELECT e *JOIN*, embora SQL e álgebra possuam diferenças importantes.

Operações fundamentais

σ seleção
Escolhe linhas por condição

π projeção
Escolhe atributos (colunas)

⋈ junção
Combina relações por condição

∪, −, ×
Operações de conjuntos e produto

Seleção na álgebra relacional

Tópico: σ escolhe linhas que atendem a uma condição

Seleção: σ

$$\sigma_{\text{cidade}=\text{'Vitória'}}(\text{Clientes})$$

SQL: SELECT * FROM clientes
WHERE cidade = 'Vitória';

A seleção é representada pela letra grega sigma, σ, e mantém apenas as tuplas que satisfazem um predicado, isto é, uma condição lógica. No banco de exemplo, podemos selecionar clientes cuja cidade seja Vitória, mantendo as mesmas colunas da relação original.

Projeção na álgebra relacional

Tópico: π escolhe atributos de uma relação

A projeção é representada por pi, π, e permite escolher quais atributos, ou colunas, aparecerão no resultado da operação. Por exemplo, projetar nome e cidade da relação Clientes elimina da apresentação as colunas id e email, preservando somente as desejadas.

Projeção: π

$$\pi_{\text{nome,cidade}}(\text{Clientes})$$

SQL: SELECT DISTINCT nome, cidade
FROM clientes;

Junção na álgebra relacional

Tópico: ⋈ combina informações de relações distintas

No estudo de caso, a condição clientes.id = pedidos.cliente_id permite relacionar o nome de uma pessoa às compras que realizou. Essa operação corresponde, de forma simplificada, ao INNER *JOIN* com ON em SQL; clientes sem pedidos não aparecem nessa junção interna.

Junção: ⋈

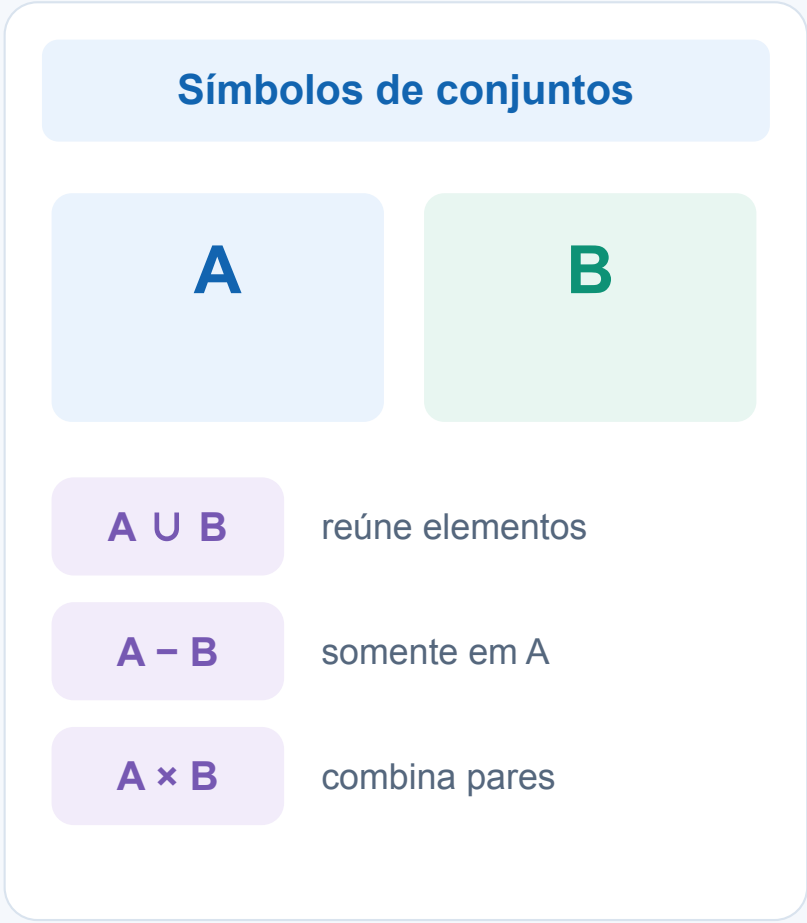
Clientes ⋈ id=cliente_id Pedidos

SQL: FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id

União, diferença e produto

Tópico: Operações de conjuntos no modelo relacional

A união reúne tuplas presentes em uma relação ou em outra, desde que os dois resultados tenham atributos compatíveis. O produto cartesiano forma combinações entre todas as tuplas de duas relações, podendo gerar muitos resultados antes de qualquer filtro ou junção.



Traduzindo álgebra para SQL

Tópico: Compor seleção, junção e projeção

Uma pergunta como quais clientes possuem pedidos pagos pode ser descrita por uma junção, seguida de seleção pelo status e projeção do nome. Em SQL, essa combinação aparece como *JOIN* para relacionar, WHERE para filtrar e SELECT DISTINCT para escolher nomes sem repetição.

Consulta equivalente

```
SELECT DISTINCT c.nome
FROM clientes AS c
JOIN pedidos AS p
 ON p.cliente_id = c.id
WHERE p.status = 'pago';

-- Ana, Bruno e Carla
```

Introdução ao cálculo relacional

Tópico: Descrever o resultado por condições lógicas

Duas abordagens

Álgebra relacional

Descreve operações sobre relações

Cálculo relacional

Descreve condições do resultado

SQL

Linguagem declarativa prática

O cálculo relacional é outra base formal para consultas: em vez de enumerar operações, ele descreve quais elementos devem satisfazer determinada condição. Por isso, costuma ser apresentado como uma abordagem declarativa, interessada no resultado desejado e não em um algoritmo passo a passo.

Cálculo relacional de tuplas

Tópico: Variáveis representam registros completos

No cálculo de tuplas, usamos variáveis que representam registros de uma relação e escrevemos uma condição que caracteriza os resultados desejados. A notação formal é apresentada de modo simplificado nesta aula; o importante é perceber a ligação entre predicado lógico e filtro SQL.

Exemplo com tuplas

$$\{ c \mid c \in \text{Clientes} \land c.\text{cidade} = \text{'Vitória'} \}$$

Ler: conjunto de tuplas c de Clientes que satisfazem a condição.

Cálculo relacional de domínios

Tópico: Variáveis representam valores de atributos

No cálculo de domínios, as variáveis representam valores individuais dos atributos, como identificador, nome, cidade e email de um cliente. A condição associa esses valores a uma tupla válida da relação e seleciona somente aqueles que satisfazem o filtro desejado.

Exemplo com domínios

$$\{ \langle n \rangle \mid \exists i, e : \text{Clientes}(i,n,\text{'Vitória'},e) \}$$

n representa o nome; i e e representam
outros valores da mesma linha.

Álgebra, cálculo e SQL

Tópico: Três maneiras de expressar uma consulta

A álgebra relacional enfatiza operações como selecionar, projetar e juntar; o cálculo relacional enfatiza condições lógicas para pertencer ao resultado. O SQL reúne ideias declarativas e recursos práticos de bancos reais, como valores NULL, agregações, ordenação e manutenção de dados.

Comparação rápida

Álgebra

σ, π, ⋈ e operações de conjunto

Cálculo

Variáveis, predicados e quantificadores

SQL

SELECT, WHERE, JOIN e DISTINCT

Exercício resolvido: visão

Tópico: Criar um relatório reutilizável de pedidos pagos

Resolução — etapa 1

```
CREATE VIEW vw_resumo_pagos AS
SELECT c.nome AS cliente,
       c.cidade, p.valor
FROM clientes c
JOIN pedidos p
 ON p.cliente_id = c.id
WHERE p.status = 'pago';

SELECT * FROM vw_resumo_pagos;
```

Problema: a loja precisa consultar frequentemente o nome do cliente, a cidade e o valor de cada pedido que já foi pago. Solução: criar uma visão que junte clientes e pedidos e aplique o filtro status igual a pago, evitando repetir a mesma consulta.

Exercício resolvido: índice e transação

Tópico: Observar a busca e desfazer uma alteração

Primeiro, crie o índice sobre status e use EXPLAIN *QUERY PLAN* para verificar a estratégia de acesso escolhida pelo SQLite. Depois, insira um pedido dentro de uma transação e execute *ROLLBACK*; o registro deve desaparecer quando a consulta for repetida.

Resolução — etapa 2

```
CREATE INDEX IF NOT EXISTS
idx_pedidos_status
ON pedidos(status);

EXPLAIN QUERY PLAN
SELECT id FROM pedidos
WHERE status = 'pago';

BEGIN;
INSERT INTO pedidos VALUES
(107,4,75,'aberto');
ROLLBACK;
```

Exercício avaliativo

Tópico: Atividade individual no DBeaver com SQLite

Com o banco de testes preparado, crie uma visão que mostre os pedidos em aberto com o nome do cliente e o valor da compra. Crie um índice para apoiar a busca por cliente_id e observe o plano de uma consulta filtrada por esse campo.

Critérios de avaliação

1. Visão correta e consulta executável
2. Índice criado e plano interpretado
3. Explicação da restrição CHECK
4. Seleção σ traduzida para SQL

Referências e materiais de apoio

Tópico: Documentação oficial e bibliografia de consulta

Os exemplos práticos foram preparados para SQLite e DBeaver, considerando particularidades como a ativação de chaves estrangeiras e as visões comuns somente de leitura. Para fundamentos matemáticos, utilize livros de sistemas de bancos de dados que apresentem álgebra relacional, cálculo de tuplas e cálculo de domínios.

Links oficiais e bibliografia

- SQLite — CREATE VIEW
- SQLite — CREATE INDEX
- SQLite — Transações
- SQLite — Chaves estrangeiras
- DBeaver — Editor SQL

Anexo - Laboratório

```sql
-- Laboratório de Sistemas de Bancos de Dados (DBeaver + SQLite)
-- Professor Msc Elionai Magalhães
-- Execute em uma conexão SQLite de testes; este script recria as tabelas.
-- ATENÇÃO: não execute em um banco que contenha dados importantes.


PRAGMA foreign_keys = ON;
DROP VIEW IF EXISTS vw_pedidos_pagos;
DROP TABLE IF EXISTS pedidos;
DROP TABLE IF EXISTS clientes;


CREATE TABLE clientes (
    id INTEGER PRIMARY KEY,
    nome TEXT NOT NULL,
    cidade TEXT NOT NULL,
    email TEXT UNIQUE
);


CREATE TABLE pedidos (
    id INTEGER PRIMARY KEY,
    cliente_id INTEGER NOT NULL,
    valor REAL NOT NULL CHECK (valor > 0),
    status TEXT NOT NULL DEFAULT 'aberto'
        CHECK (status IN ('aberto',
'pago', 'cancelado')),
    FOREIGN KEY (cliente_id) REFERENCES
clientes(id)
);
```

```sql
INSERT INTO clientes (id, nome, cidade,
email) VALUES
(1, 'Ana', 'Vitória', 'ana@exemplo.com'),
(2, 'Bruno', 'Serra',
'bruno@exemplo.com'),
(3, 'Carla', 'Vila Velha',
'carla@exemplo.com'),
(4, 'Diego', 'Cariacica',
'diego@exemplo.com');


INSERT INTO pedidos (id, cliente_id,
valor, status) VALUES
(101, 1, 120.00, 'pago'),
(102, 1,  80.00, 'aberto'),
(103, 2, 200.00, 'pago'),
(104, 3,  50.00, 'cancelado'),
(105, 3, 150.00, 'pago'),
(106, 2,  90.00, 'aberto');


-- Inspeção inicial:
SELECT * FROM clientes ORDER BY id;
SELECT * FROM pedidos ORDER BY id;


-- Visão: consulta nomeada; o resultado
não é armazenado separadamente.
CREATE VIEW IF NOT EXISTS
vw_pedidos_pagos AS
SELECT p.id AS pedido_id, c.nome AS
cliente, c.cidade,
       p.valor, p.status
FROM pedidos AS p
JOIN clientes AS c ON c.id = p.cliente_id
WHERE p.status = 'pago';
```

```sql
-- Observação: o otimizador pode escolher
SCAN em tabelas muito pequenas.


-- Índices também consomem espaço e têm
custo de manutenção.


-- Verificação das restrições:
PRAGMA foreign_keys;
PRAGMA foreign_key_list(pedidos);
-- Testes opcionais (executar
individualmente para observar erros):
-- INSERT INTO pedidos (id, cliente_id,
valor) VALUES (107, 999, 40);
-- INSERT INTO pedidos (id, cliente_id,
valor) VALUES (108, 1, -10);
-- INSERT INTO clientes (id, nome,
cidade, email)
-- VALUES (5, 'Eva', 'Vitória',
'ana@exemplo.com');


-- Demonstração de transação: inserir e
desfazer, mantendo os dados originais.
BEGIN TRANSACTION;
INSERT INTO pedidos (id, cliente_id,
valor, status)
VALUES (107, 4, 75.00, 'aberto');
SELECT * FROM pedidos WHERE id = 107;
ROLLBACK;
SELECT * FROM pedidos WHERE id = 107; --
nenhum registro
```

```sql
-- Para confirmar uma alteração, execute
separadamente:
-- BEGIN TRANSACTION;
-- UPDATE pedidos SET status = 'pago'
WHERE id = 102;
-- COMMIT;


-- Exercício resolvido: clientes e
valores dos pedidos pagos.
SELECT cliente, SUM(valor) AS total_pago
FROM vw_pedidos_pagos
GROUP BY cliente
ORDER BY total_pago DESC;


-- Álgebra relacional: seleção (σ),
projeção (π) e junção (⋈)
-- Equivalente SQL: nomes dos clientes
que possuem pedidos pagos.
SELECT DISTINCT c.nome
FROM clientes AS c
JOIN pedidos AS p ON p.cliente_id = c.id
WHERE p.status = 'pago';
```